

Vanity Errors - Quick Debugging Guide

Cookie Clicker | React + TypeScript | What broke, why it broke, and how to spot it next time

A useful debugging order: read the error message -> find the underlined identifier -> check its type and scope -> fix the cause, not every follow-on error.

1 Type mismatch: [] vs number[] (TS2345 / TS7006)

You declared `ownedVanityIds: []` as a type. That means an empty tuple which cannot hold any numbers. Your purchase logic adds an ID and TypeScript rejects it.

```
// GameSave.ts
ownedVanityIds: number[]; // TYPE: any number of IDs

// defaultSave
ownedVanityIds: [], // VALUE: starts empty
```

Recognize it: Errors such as "[number] is not assignable to []" or "number is not assignable to never" usually mean an array was typed too narrowly.

2 Unknown data: validate before using .every() (TS18046)

`JSON.parse` returns untrusted data. First check that the property exists, then narrow it to an array with `Array.isArray()`, then inspect its elements using `.every()`.

```
// Inside isGameSave(value: unknown): value is GameSave
if (typeof value !== "object" || value === null) return false;
if (!("ownedVanityIds" in value)) return false;

return Array.isArray(value.ownedVanityIds) &&
  value.ownedVanityIds.every(
    (id: unknown) => typeof id === "number" &&
      Number.isInteger(id) && id > 0
  );
```

Recognize it: When TypeScript says "is of type unknown", check the runtime type before calling methods. Keep your existing cookie-number validation alongside these conditions.

3 Function-call typo: Array.isArray versus .every()

You accidentally wrote `value.ownedVanityIds(...)`, as if the array were a function. These are two separate checks:

```
Array.isArray(value.ownedVanityIds) &&
value.ownedVanityIds.every((id: unknown) => Number.isInteger(id))
```

Recognize it: An error near parentheses or "not callable" often means you forgot a method name such as `.every`, or placed brackets incorrectly.

Vanity Component - Props, Scope & JSX

Most of these errors came from data not being available where you tried to use it.

4 Cannot find name: ownedVanityIds (TS2304)

The Vanity page does not automatically see state owned by App. Include the array in the component props type, receive it in the parameter list, and pass it from App.

```
// Endgameitems.tsx
type VanityProps = {
  cookies: number;
  vanity: Progression[];
  ownedVanityIds: number[];
  onBuyVanity: (item: Progression) => void;
};

export function Vanity({ cookies, vanity, ownedVanityIds,
  onBuyVanity }: VanityProps) { /* ... */ }

// App.tsx (Route's element)
<Vanity cookies={cookies} vanity={vanity}
  ownedVanityIds={game.ownedVanityIds}
  onBuyVanity={buyVanity} />
```

Recognize it: If you see "Cannot find name X", ask: Is X defined in this function? Imported? Received as a prop? Is the capitalization identical?

5 Cannot find name: item (TS2304)

item only exists inside the callback that receives it. Calculate ownership inside vanity.map((item) => { ... }), not above the map.

```
{vanity.map((item) => {
  const owned = ownedVanityIds.includes(item.id);
  return (
    <article key={item.id}>
      <h2>{item.name}</h2>
      <button disabled={owned || cookies < item.cost}
        onClick={() => onBuyVanity(item)}>
        {owned ? "Owned" : "Buy"}
      </button>
    </article>
  );
})}
```

Recognize it: Variables declared inside a callback are scoped to that callback. Look for where the variable was introduced.

6 JSX bracket mismatch: closing .map()

With a block-bodied map callback, finish the returned JSX, close the callback, close map(), then close the JSX expression. The ending is); } } across the final lines (as shown above).

Recognize it: If the compiler highlights an unexpected brace or parenthesis, match each opening {, (and JSX tag with its closing partner.

7 Spelling: ownedVanityids vs ownedVanityIds

Your App used ownedVanityids (lowercase i) even though the property is ownedVanityIds (uppercase I). You also had no standalone variable by that name.

```
// Simplest and clearest in App.tsx:  
ownedVanityIds={game.ownedVanityIds}
```

Recognize it: TypeScript names are case-sensitive. Try dot-autocomplete (game.) before creating another import or variable.

Quick Checklist & Final Code Pattern

Use this small diagnostic sequence when VS Code shows a red underline.

Error wording	First check
Cannot find name X	Typo? Wrong scope? Missing prop? Missing import?
X is of type unknown	Add a type guard: <code>typeof</code> , <code>in</code> , <code>Array.isArray</code> .
Type A not assignable to B	Compare the exact declared type and returned value.
number not assignable to never	An array/tuple may be typed as <code>[]</code> .
Unexpected <code>}</code> or <code>)</code>	Match braces and JSX <code>.map()</code> endings.
Property does not exist	Check spelling/capitalization and the type definition.

Correct purchase flow

The App owns the saved state; Vanity only receives props and displays ownership. Keep the purchase checks inside the state updater so each update uses the latest state.

```
function buyVanity(item: Progression) {  
  setGame(current => {  
    if (current.ownedVanityIds.includes(item.id)) return current;  
    if (current.cookies < item.cost) return current;  
    return {  
      ...current,  
      cookies: current.cookies - item.cost,  
      ownedVanityIds: [...current.ownedVanityIds, item.id],  
    };  
  });  
}
```

Three debugging habits that save time

1. Fix the first underlying error, then recheck; several later errors may disappear.
2. Hover the red underline and read the exact identifier and expected type.
3. Run `npm run check` (if your project defines it) to verify TypeScript without starting the game.

Remember: type = what data is allowed; props = what a component receives; scope = where a name exists.